



Java Arrays Summary

An array in Java is an object that holds a **fixed number** of values of a **single data type**. It is essentially a container used to store multiple elements of the same type in adjacent memory locations.

Key Characteristics

1. **Fixed Size:** Once an array is created, its size cannot be changed.
2. **Zero-Indexed:** Array elements are accessed using a numbered index, starting at 0.
3. **Homogeneous:** An array can only store elements of the specific type it was declared with (e.g., an `int[]` can only hold integers).

1. Array Declaration and Initialization

There are two primary ways to declare and initialize an array in Java:

Method A: Declaring and Allocating Size (Empty)

This method creates an array with a specified size, but the contents are initialized to default values (e.g., 0 for int, null for String).

Concept	Syntax	Example
Declaration	<code>dataType[] arrayName;</code>	<code>int[] numbers;</code>
Allocation	<code>arrayName = new dataType[size];</code>	<code>numbers = new int[5];</code>
Combined	<code>int[] numbers = new int[5];</code>	Creates an array of 5 integers (index 0 to 4).

Method B: Declaring and Initializing Values (Literal)

This method creates the array and fills it with initial values in one step. The size is automatically determined by the number of elements provided.

Concept	Syntax	Example
Initialization	<code>dataType[] arrayName = {value1, value2, ...};</code>	<code>String[] names = {"Alice", "Bob", "Charlie"};</code>

2. Accessing and Modifying Elements

Elements are accessed using the index operator [].

```
public class ArrayExample {
    public static void main(String[] args) {
        // Creates an array of 3 names
        String[] names = {"Alice", "Bob", "Charlie"};

        // Accessing an element (Index starts at 0)
        String firstElement = names[0]; // "Alice"
        System.out.println("First name: " + firstElement);

        // Modifying an element
        names[1] = "Bobby";
        System.out.println("Modified name: " + names[1]);

        // Accessing the length of the array
        int arraySize = names.length; // 3
        System.out.println("Array size: " + arraySize);

        // Trying to access names[3] would cause an ArrayIndexOutOfBoundsException
    }
}
```

3. Introduction to ArrayList (Dynamic Array)

Unlike standard arrays, the ArrayList class is part of the Java Collections Framework and provides a **resizable array** implementation. This is the preferred choice when you don't know how many elements you will need.

Key Differences from Arrays

- **Dynamic Size:** The size automatically grows and shrinks as elements are added or removed.
- **Object Storage:** It can only store **objects** (e.g., String, Integer, Double), not primitive types (like int, byte). This is why we use the Wrapper Class Integer instead of int.

ArrayList Usage Example

```
import java.util.ArrayList;
```

```
public class ArrayListExample {
    public static void main(String[] args) {
        // Declaration: Must specify the object type in angle brackets (<>)
        // Notice we use 'String', which is an object.
        ArrayList<String> cities = new ArrayList<>();

        // 1. Adding elements (Dynamically grows the size)
        cities.add("New York");
        cities.add("London");
        cities.add("Paris"); // Size is now 3

        // 2. Accessing elements (uses .get(index))
        String secondCity = cities.get(1); // "London"
        System.out.println("Second City: " + secondCity);

        // 3. Getting the current size (uses .size() instead of .length)
        int currentSize = cities.size(); // 3
        System.out.println("Current Size: " + currentSize);

        // 4. Removing an element (Automatically shrinks the list)
        cities.remove(0);
        System.out.println("New First City: " + cities.get(0)); // "London"
    }
}
```